AI | GENERATIVE AI | LLM SOLUTIONS ARCHITECTURE

LLM ENGINEERING MASTERY

The Complete Field Guide for Generative AI Engineer, LLM Solutions Architect & AI/ML Engineer Roles

Architecture | RAG | Agents | LangChain | LangGraph | MLOps | AWS/GCP | FAANG Interview Prep | Python

WHAT'S INSIDE

- End-to-end LLM & Transformer architecture
- RAG, vector DBs, LangChain, LangGraph
- AI Agents & multi-agent systems
- Production LLMOps, AWS SageMaker, GCP Vertex AI
- Scaling, cost optimization, troubleshooting
- Executable Python code + Python interview Q&A throughout
- Meta / Google / Amazon / Microsoft / Tesla / Apple interview styles
- 5 company practice projects (Meta, Google, Microsoft, Tesla, Apple)
- Scenario-based & rapid-fire interview Q&A

Deepika Chintala
Senior AI/ML Engineer | Generative AI & LLM Solutions
linkedin.com/in/deepikachintala

Study & Interview Prep Edition
2026

TOPIC 01 • FOUNDATIONS

LLM Fundamentals

What a Large Language Model actually is, how it generates text, and when to reach for one in production.

CONCEPT What It Is & How It Works

An LLM is a neural network (usually a **decoder-only Transformer**) trained on massive text corpora to predict the **next token** given prior context. Through next-token prediction at scale, it learns grammar, facts, reasoning patterns and style — then at inference time it autoregressively samples tokens one at a time, feeding each output back in as input.

- **Pre-training**: self-supervised next-token prediction on trillions of tokens (web, code, books).
- **Fine-tuning / SFT**: supervised examples teach instruction-following.
- **RLHF / DPO**: aligns outputs to human preference (helpful, harmless, honest).
- **Inference**: tokenize → embed → transformer layers → softmax over vocabulary → sample next token → repeat.

Key Point: An LLM doesn't "know" facts the way a database does. It compresses statistical patterns from training data into weights, and that's exactly why hallucination, knowledge cutoffs, and the need for RAG or grounding exist. Interviewers love probing this one.

Key Point: The context window is short-term memory, nothing more. Once a call ends, nothing carries over unless you build memory yourself (conversation history, a vector store, session state).

Key Point: Temperature, top-p, and top-k all control how random the output is. Keep it low for factual work, push it higher when you want more creative or varied answers.

DECISION When / How / Where to Use

Scenario	Use an LLM?	Why
Summarization, drafting, classification, code-gen	Yes	Language understanding + generation is the core strength
Exact math, deterministic business rules	No / Tool-augmented	LLMs approximate; delegate to a calculator/rules engine via function calling
Private / proprietary knowledge Q&A	Yes, with RAG	Ground answers in retrieved documents, not parametric memory
Real-time transaction decisions (fraud block)	No (or as one signal)	Latency + non-determinism unsuitable as sole decision-maker in regulated flow

TIPS Production Reality Checklist

Cost per token | Latency (TTFT/TPS) | Context limits | Hallucination risk | PII leakage | Rate limits | Model drift on upgrade | Prompt injection

PREP Interview Q&A

Q: How does an LLM actually generate the next word? CORE

It computes a probability distribution over the vocabulary using softmax on the final transformer layer's output. That distribution is conditioned on all prior tokens through self-attention, and then a decoding strategy (greedy, top-k, top-p, temperature-scaled) picks the actual next token.

Q: Why do LLMs hallucinate, and how do you mitigate it in production? SENIOR

Because the model is optimizing for what sounds plausible, not for what's verified true. In practice I lean on RAG grounding, prompts that force citations, output validation and guardrails, lower temperature for factual tasks, and human review for anything high-stakes.

Q: When would you NOT use an LLM for a task? SENIOR

Anywhere you need deterministic, auditable logic. Interest calculations in banking, sub-50ms latency requirements, or anything that needs 100% accuracy guarantees. I'd hand those off to rules engines or traditional ML and only bring the LLM in for the unstructured-language parts.

Q: What's the difference between pre-training and fine-tuning? CORE

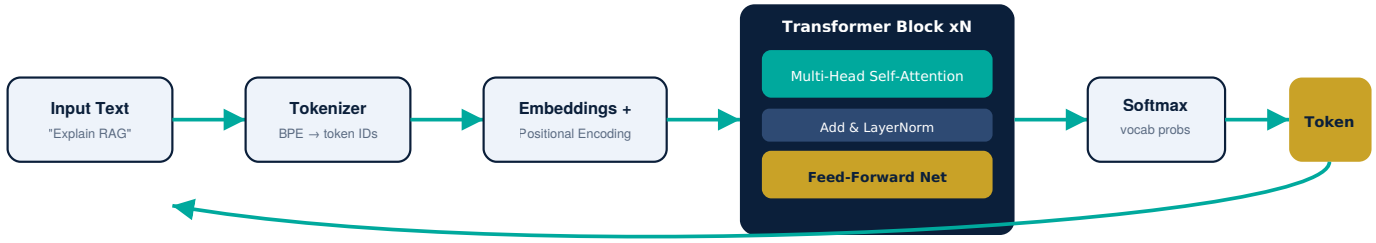
Pre-training is where the model learns general language ability, through self-supervised learning on huge unlabeled text. Fine-tuning comes after, on a smaller labeled dataset specific to your task or domain, and it shifts the model's behavior toward instructions or a particular use case (full fine-tune or via LoRA/PEFT).

TOPIC 02 • ARCHITECTURE

Transformer Architecture End-to-End

The block every modern LLM (GPT, Claude, Llama, Gemini) is built from.

DIAGRAM End-to-End Flow



Autoregressive loop: predicted token is appended and fed back in until an end-of-sequence token is produced

KEY POINTS Core Mechanics

- **Self-Attention**: each token computes Query/Key/Value vectors; attention scores = softmax($QK^T/\sqrt{d}$) determine how much every other token influences it.
- **Multi-Head**: attention is split into parallel heads so the model learns different relationship types (syntax, coreference, long-range dependency) simultaneously.
- **Positional Encoding**: attention has no inherent order awareness, so position info (sinusoidal or RoPE) is injected.
- **Residual connections + LayerNorm** stabilize training of very deep stacks (GPT-4 class models: 100+ layers).
- **Feed-Forward Network** per block adds non-linearity/capacity, applied identically per token position.
- **Decoder-only vs Encoder-Decoder**: GPT/Claude/Llama are decoder-only (causal mask, generation-focused); T5/BART are encoder-decoder (translation/summarization-focused).

PREP Interview Q&A

Q: Walk me through what happens inside one transformer block. SENIOR

Input embeddings and positional encoding go into multi-head self-attention first, so each token can weigh every other token through scaled dot-product attention. That result gets added back as a residual and normalized, then run through a position-wise feed-forward network with its own residual and norm. Stack that N times, and a final softmax turns the last layer's output into vocabulary logits.

Q: Why is attention 'quadratic' and why does that matter in production? SENIOR

Because attention builds an NxN score matrix for a sequence of length N, compute and memory both scale quadratically as context grows. That's the real reason long-context inference gets expensive, and it's why FlashAttention, KV-caching, and sliding-window or sparse attention all exist in serious serving stacks.

PRODUCTION Real-World Example & AI/ML Engineer Tips

Real-world example: I once had a team come to me confused because GPT-style responses were degrading past roughly 8K tokens of chat history. It wasn't a bug. It was attention dilution over a long context, and the fix was simply summarizing older turns into a rolling memory instead of replaying the full raw history on every call.

Advantage to know: Transformers train in parallel, unlike RNNs or LSTMs which have to process sequentially. That's the whole reason they scale so well on GPU and TPU clusters, and it's a favorite "why did Transformers win" interview question.

Interview tip: If someone asks how you'd speed up inference, lead with KV-caching (it avoids recomputing attention for tokens you've already generated) before you jump to quantization or hardware. It signals you actually understand the architecture, not just the tooling around it.

- **Use case:** Long-document summarization needs models with extended context (128K+) or a chunk-and-reduce map-reduce pattern.
- **Tip:** RoPE (rotary position embeddings) generalizes to longer sequences far better than fixed sinusoidal encoding. Worth knowing that trade-off cold.

TOPIC 03 • FOUNDATIONS

Tokenization & Embeddings

The layer that decides your cost, your context budget, and your retrieval quality.

CONCEPT Key Points

- Tokenizers (BPE/tiktoken, SentencePiece) split text into sub-word units, not whole words — 1 word ≠ 1 token (~4 chars/token in English).
- Embeddings map tokens/sentences into dense vectors (e.g. 1536-dim for text-embedding-3-small) capturing semantic meaning — similar meaning = close vectors (cosine similarity).
- Context window = max tokens (input+output) a model can process; exceeding it truncates or errors.

Production tip: Always count tokens before sending a request (tiktoken) to budget cost and avoid truncation surprises — a common on-call incident is "response cut off mid-JSON" caused by hitting max_tokens.

Production tip: Use a smaller/cheaper embedding model for high-volume retrieval and reserve the large chat model only for generation — this alone can cut GenAI pipeline cost 40-60%.

CODE Code: Token Counting & Embeddings

```
import tiktoken
from openai import OpenAI

client = OpenAI()
enc = tiktoken.encoding_for_model("gpt-4o")

# 1. Count tokens BEFORE calling the API - avoids cost surprises & truncation
text = "Explain how RAG grounds LLM answers in enterprise documents."
n_tokens = len(enc.encode(text))
print(f"Prompt tokens: {n_tokens}")   # budget check against context window

# 2. Generate an embedding for semantic search / RAG indexing
resp = client.embeddings.create(
    model="text-embedding-3-small",   # cheap, 1536-dim, good recall
    input=text
)
vector = resp.data[0].embedding          # list[float], len == 1536
print(len(vector))
```

PREP Interview Q&A

Q: Why can two semantically identical sentences have different token counts? CORE

It comes down to subword splitting, not meaning. Rare words, numbers, code, and non-English text all fragment into more tokens, simply because the BPE vocabulary was built on frequency, so uncommon strings get chopped into smaller pieces.

Q: How do you choose an embedding dimension/model for a production RAG system? SENIOR

It's a balance between retrieval quality and storage or latency cost. Smaller dims, say 384 to 768, are cheaper to store and index and fast enough for most enterprise corpora. I only move up to 1536 or 3072 when recall on ambiguous domain language genuinely improves in eval, and even then I benchmark against our own labeled query set rather than trusting a leaderboard.

PRODUCTION Real-World Example

Real-world example: A support chatbot's cost jumped 3x almost overnight after a prompt update. What happened was the new prompt embedded the full conversation history on every embeddings call for semantic caching, which tripled token volume without anyone noticing at first. The fix was straightforward once we found it: truncate and summarize history before embedding, and only embed the latest user turn for cache lookups.

TOPIC 04 • CORE SKILL

Prompt Engineering for Production

The cheapest lever you have before touching model weights or infra.

KEY POINTS Patterns That Work in Production

Pattern	Use When	Example
Zero-shot	Simple, well-understood task	"Classify sentiment: ..."
Few-shot	Format/style must be exact	3 input→output examples before the real query
Chain-of-Thought	Multi-step reasoning, math, planning	"Think step by step before answering"
ReAct	Needs tools/external data mid-reasoning	Thought → Action → Observation loop
Structured Output	Downstream code consumes the result	JSON schema / function-calling / Pydantic

CODE Production Prompt Template

```
# System prompt structure used in a banking document-QA assistant
SYSTEM = (
    "You are a banking compliance assistant.\n"
    "ROLE: Answer only from the provided CONTEXT.\n"
    "RULES:\n"
    "1. If not in CONTEXT, say 'I don't have that information.'\n"
    "2. Never fabricate account numbers, rates, or policy clauses.\n"
    "3. Cite the source section for every claim.\n"
    "OUTPUT FORMAT: JSON {answer, citations, confidence}"
)

def build_prompt(context, question):
    return [
        {"role": "system", "content": SYSTEM},
        {"role": "user", "content": f"CONTEXT:\n{context}\n\nQUESTION: {question}"}
    ]
```

PREP Interview Q&A

Q: How do you prevent prompt injection in a customer-facing app? SENIOR

Keep trusted system instructions completely separate from untrusted user or document content, and never let retrieved content redefine the system role. I also validate and sanitize tool outputs before feeding them back in, use an output parser with strict schemas, and put a guardrail model or regex check in front of any high-risk action like moving money or disclosing PII.

Q: Few-shot vs fine-tuning — how do you decide? SENIOR

I default to few-shot and prompt engineering first since it's fast, cheap, and easy to walk back. Fine-tuning only earns its place when I've got a large, stable labeled dataset, need to lock in a fixed style or format at low per-call token cost, or need behavior that prompting genuinely can't achieve. Even then, I A/B it against a strong prompted baseline before shipping anything.

PRODUCTION Real-World Example & Interview Tips

Real-world example: A loan-summary prompt looked flawless in testing, then produced inconsistent JSON in production for about 2% of cases. Switching from free-text JSON instructions to the provider's native structured-output/function-calling mode fixed it for good, since that enforces the schema at the API level instead of hoping the model complies every time.

Interview tip: When someone asks you to design a prompt, mention how you'd *test* it, meaning a golden set and an eval score. That's usually the difference interviewers are listening for between someone who treats prompting as iterative and measurable versus someone guessing once and hoping.

TOPIC 05 • DECISION FRAMEWORK

Fine-Tuning vs RAG vs Prompt Engineering

The #1 architecture decision interviewers probe — get this crisp.

FRAMEWORK Decision Matrix

Need	Prompting	RAG	Fine-tuning
Fresh / frequently changing knowledge	No	**Best**	No (stale fast)
Company-proprietary documents	No	**Best**	Risky (leak/retrain cost)
Fixed output style / tone / format	OK	OK	**Best**
Domain vocabulary (legal, banking, medical)	OK	Good	**Best**
Lowest cost to start	**Best**	Medium	Highest (data+compute)
Need citations / traceability	No	**Best**	No

TIPS Rule of Thumb I Use On the Job

My default order is prompting first, then RAG for knowledge grounding, and fine-tuning (usually LoRA or QLoRA, rarely a full fine-tune) only once RAG and prompting have plateaued on style, format, or latency, or when I need to shrink a large model into something smaller and cheaper. In banking, that played out as prompt engineering for policy Q&A, RAG for grounding against internal policy PDFs, and LoRA fine-tuning reserved for one structured-extraction model where the output format had to be completely consistent at high volume.

PREP Interview Q&A

Q: A client wants the model to 'know' their entire product catalog that updates daily. RAG or fine-tune? SENIOR

RAG, without question. A catalog that changes daily would make fine-tuning stale within 24 hours and mean constant retraining cost. I'd index the catalog in a vector store with a nightly or streaming ingestion job and retrieve the top-k relevant items per query, so the model stays current without ever needing to be retrained.

Q: When have you chosen fine-tuning over RAG? SENIOR

One case that stands out: a fixed, compact JSON extraction schema running at high throughput and low latency. RAG's extra retrieval hop added latency, and the prompt-engineered version kept being inconsistent on edge-case formatting. A LoRA fine-tune on around 5,000 labeled examples locked the format in reliably and cut prompt token cost too, since the instructions no longer had to be spelled out on every call.

PRODUCTION Real-World Examples for AI/ML Engineers

- **Use case:** Customer support bot → RAG over product docs, because answers must stay current as products change weekly.
- **Use case:** Legal contract clause classifier → fine-tuned small model, because the label set is fixed and volume is high (thousands/day).
- **Advantage:** Combining both often beats either alone. Fine-tuning teaches format and tone, and RAG supplies the facts.

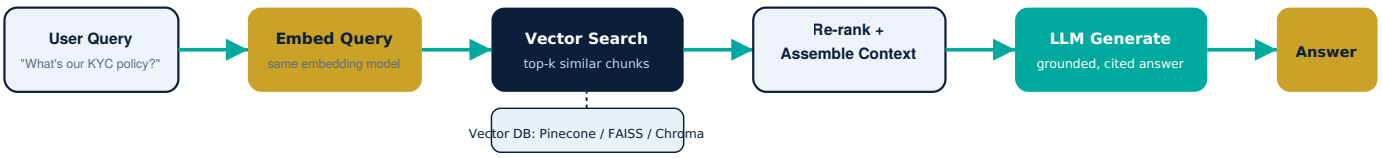
Interview tip: Don't ever answer this one with just "it depends." Walk through the decision matrix out loud instead (freshness, proprietary data, cost, latency, traceability) so the interviewer sees your framework, not just where you landed.

TOPIC 06 • GROUNDING

RAG Architecture (Condensed)

Full deep-dive lives in the companion RAG Architecture document — this page is the interview-ready summary.

DIAGRAM End-to-End Flow



CONCEPT Key Points

- **Ingestion**: chunk documents (semantic or fixed-size w/ overlap), embed, store with metadata.
- **Retrieval**: embed query, ANN search (cosine/dot), optionally hybrid (BM25 + vector) + re-ranker (cross-encoder).
- **Generation**: inject retrieved chunks into the prompt as grounding context; force citations.
- **Advanced patterns**: Agentic RAG (agent decides what/when to retrieve), Graph RAG (knowledge graph traversal), Self-RAG (model critiques its own retrieval need).
- **Chunk size trade-off**: too small → loses context; too large → dilutes relevance and wastes tokens.
- **Evaluation**: measure retrieval (recall@k, MRR) and generation (faithfulness, answer relevance) separately.

PREP Interview Q&A

Q: Your RAG chatbot gives confident wrong answers. How do you debug it? SENIOR

I always split retrieval from generation before touching anything else. First I check whether the correct chunk was even retrieved. If it wasn't, that's a chunking or embedding or indexing problem. If the right chunk was there and the answer is still wrong, that's on the generation side, so I tighten it with explicit 'answer only from context' instructions, lower temperature, and citation-forcing.

Q: How would you keep a RAG system's index fresh for a knowledge base that updates hourly? SENIOR

I'd set up an incremental ingestion pipeline, either CDC or webhook-triggered, that only re-embeds changed or new documents. Upsert instead of a full reindex, version the chunk IDs so stale entries get invalidated properly, and run a lightweight nightly consistency job to catch anything that drifted.

PRODUCTION Real-World Examples & Tips

- **Use case:** Internal HR policy bot — RAG over an HR handbook, cutting ticket volume to the HR team significantly.
- **Use case:** Customer-facing product Q&A — RAG over product manuals with strict 'don't answer if not found' guardrails.

Interview tip: When you're asked to design a RAG system, state your chunking strategy and size out loud, something specific like "500 tokens with 50 overlap, semantic splitting." A vague "I'd chunk the documents" reads as under-prepared to anyone senior.

TOPIC 07 • INFRASTRUCTURE

Vector Databases in Production

Pinecone, FAISS, Chroma, Weaviate — how to pick and operate one at scale.

KEY POINTS Comparison

DB	Best For	Notes
FAISS	Local/offline, full control	Library not a service — you manage persistence, scaling, HA yourself
Pinecone	Managed, production SaaS	Serverless scaling, metadata filtering, low ops burden
Chroma	Prototyping, small-medium apps	Simple API, easy local dev, can run embedded
Weaviate / Milvus	Large-scale, hybrid search	Self-hosted or managed, strong hybrid (BM25+vector) support

CODE Code: Index + Query with Pinecone

```
from pinecone import Pinecone
from openai import OpenAI

pc = Pinecone(api_key="...")
index = pc.Index("banking-policies")
client = OpenAI()

# 1. Embed and upsert a document chunk with metadata (for filtering)
emb = client.embeddings.create(model="text-embedding-3-small", input=chunk_text).data[0].embedding
index.upsert(vectors=[{
    "id": "policy-kyc-12", "values": emb,
    "metadata": {"source": "KYC_Policy_2026.pdf", "dept": "compliance"}
}])

# 2. Query with metadata filter — narrows search to compliance docs only
q_emb = client.embeddings.create(model="text-embedding-3-small", input="KYC document
requirements").data[0].embedding
results = index.query(vector=q_emb, top_k=5, filter={"dept": {"$eq": "compliance"}}, include_metadata=True)
```

PREP Interview Q&A

Q: How do ANN indexes like HNSW avoid brute-force comparison against every vector? SENIOR

HNSW builds a multi-layer navigable small-world graph. Search starts at a sparse top layer and greedily descends into denser layers, so it only visits a small neighborhood instead of comparing against all N vectors. You trade a bit of recall for search time that's closer to logarithmic than the O(N) you'd get with brute force.

Q: Your vector search latency spiked in production — what do you check? SENIOR

A few usual suspects: index size that's grown past what the current sharding handles, an embedding dimension mismatch pushing queries onto a slower fallback path, missing metadata filter indexes causing full scans, noisy-neighbor load on a shared managed cluster, and top_k or over-fetching before re-ranking creeping up without anyone noticing.

PRODUCTION Real-World Example & Tips for AI/ML Engineers

Real-world example: I worked on a migration from FAISS running locally to Pinecone once traffic outgrew what a single-node index could reliably serve. The migration script re-embedded everything in batches with rate-limit backoff, and we had to re-validate every metadata filter since Pinecone's filter syntax doesn't match FAISS's manual post-filtering approach.

- **Advantage:** Metadata filtering lets you scope search (by department, date, tenant) without separate indexes per segment.
- **Tip:** Benchmark recall@k on your own data before picking a DB. Marketing benchmarks rarely match your actual embedding model and corpus.

TOPIC 08 • FRAMEWORK

LangChain in Production

Chains, memory, tools, retrievers — the glue layer for LLM apps.

CONCEPT Key Points

- **LCEL (LangChain Expression Language)**: compose components with the `|` pipe operator into a runnable chain.
- **Retrievers** wrap any vector store behind a common interface for RAG chains.
- **Memory** (buffer, summary, entity) persists conversation state across turns.
- **Output parsers** (Pydantic/JSON) turn free text into validated structured objects your backend can trust.

CODE Code: RAG Chain with LCEL

```
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_pinecone import PineconeVectorStore
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

vectorstore = PineconeVectorStore(index_name="banking-policies", embedding=OpenAIEmbeddings())
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

prompt = ChatPromptTemplate.from_template(
    "Answer ONLY from context.\nContext: {context}\nQuestion: {question}")
llm = ChatOpenAI(model="gpt-4o", temperature=0)

# LCEL pipe: retriever feeds context, prompt formats it, llm generates, parser extracts text
chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)
answer = chain.invoke("What documents are required for KYC?")
```

PREP Interview Q&A

Q: Why use LangChain instead of calling the OpenAI SDK directly? SENIOR

Honestly, for a single simple call I often skip it. Where it earns its place is multi-step pipelines with retrieval, multiple tools, memory, and parsing all in play. It gives me standardized interfaces so I can swap vector stores, models, or retrievers without rewriting orchestration logic, and LangSmith's built-in tracing makes debugging chains in production much less painful.

Q: A LangChain chain works in dev but times out under load — what's your approach? SENIOR

First thing I check is whether synchronous calls are blocking the event loop, and switch to async .ainvoke or .abatch if so. Then I look at connection pooling to the vector DB, whether repeated embeddings are being cached at all, and add streaming plus per-step timeouts and retries with backoff so one slow tool call can't hang the entire chain.

PRODUCTION Real-World Examples & Tips

- **Use case:** Internal analyst copilot chaining SQL-generation → execution → natural-language summary in one LCEL pipeline.
- **Use case:** Document-review chain: extract → validate → summarize, each step a swappable LangChain component.

Interview tip: If someone asks whether you've used LangChain in production, walk through an actual pipeline, the steps, why LCEL helped, one issue you ran into. That reads as hands-on experience. Listing framework features reads as something you read about last night.

TOPIC 09 • ORCHESTRATION

LangGraph for Multi-Step & Multi-Agent Flows

State machines for LLM apps — the framework that replaced fragile chained if/else logic.

CONCEPT Key Points

- LangGraph models an app as a **graph of nodes** (each a function or LLM call) and **edges** (conditional routing), with a shared, typed **state** object passed between nodes.
- Supports **cycles** (unlike plain LCEL chains) — essential for agent loops that retry, re-plan, or loop until a condition is met.
- Built-in **checkpointing** persists state so long-running or human-in-the-loop workflows can pause and resume.
- Ideal for: multi-agent handoff, approval workflows, self-correcting pipelines (generate → critique → revise).

CODE Code: Simple Self-Correcting Graph

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class State(TypedDict):
    query: str; draft: str; score: float

def generate(state: State) -> State:
    state["draft"] = llm.invoke(state["query"]).content
    return state

def critique(state: State) -> State:
    state["score"] = score_answer(state["draft"])   # e.g. faithfulness check
    return state

def route(state: State) -> str:
    return "end" if state["score"] > 0.8 else "generate"   # loop back if low quality

g = StateGraph(State)
g.add_node("generate", generate); g.add_node("critique", critique)
g.set_entry_point("generate")
g.add_edge("generate", "critique")
g.add_conditional_edges("critique", route, {"end": END, "generate": "generate"})
app = g.compile()
result = app.invoke({"query": "Summarize Q3 risk exposure"})
```

PREP Interview Q&A

Q: Why choose LangGraph over a plain LangChain chain for an agent? SENIOR

Chains built with LCEL are directed and acyclic, which is great for linear RAG but breaks down for agents. Agents need loops for retrying, re-planning, or multi-turn tool use, plus conditional branching based on whatever the runtime state looks like. LangGraph's explicit state machine gives me controllable cycles, checkpointing for human-in-the-loop steps, and debugging that's actually manageable since every node's state transition is inspectable.

PRODUCTION Real-World Example & Tips

Real-world example: I saw a document-review workflow get stuck looping forever because the critique node's score would never cross the threshold on malformed inputs. Adding a hard max-iteration count to the state, plus a fallback "escalate to human" edge, fixed it. Every production LangGraph app needs that safety net, not just a clean happy-path loop.

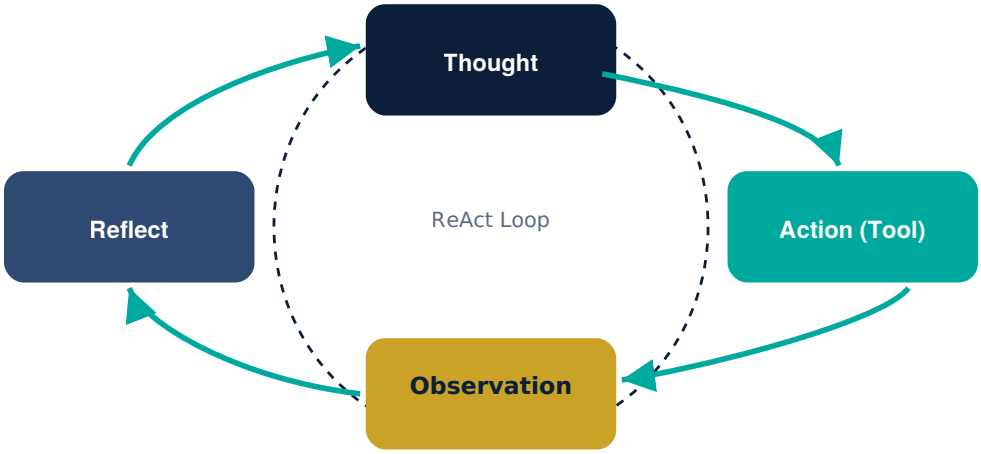
- **Use case:** Multi-step approval workflow (draft → legal review → compliance check → send), with checkpointing so a paused approval can resume days later.
- **Interview tip:** Bring up checkpointing and persistence by name. That's the feature that most separates LangGraph from a hand-rolled while-loop.

TOPIC 10 • AGENTIC AI

AI Agents (ReAct & Tool Use)

How an LLM goes from 'answers questions' to 'takes actions'.

DIAGRAM The ReAct Loop



CONCEPT Key Points

- An **agent** = LLM + tools + a loop that lets the model decide which tool to call, observe the result, and decide the next step (vs a fixed pipeline).
- **Function/Tool calling**: the model outputs structured JSON describing which function to invoke with which args; your code executes it and returns the result as an observation.
- **Planning** patterns: ReAct (reason+act interleaved), Plan-and-Execute (plan all steps upfront, then execute).

Production tip: Always set a max-iteration or max-tool-call cap plus a timeout. An ungoverned agent loop is the number one cause of runaway cost incidents in GenAI production systems.

Production tip: Log every Thought, Action, and Observation step. That trace is your only real debugging tool when an agent does something unexpected, and it's what compliance will ask for in banking anyway.

CODE Code: Tool-Calling Agent (OpenAI function calling)

```
tools = [{
    "type": "function",
    "function": {"name": "get_account_balance", "description": "Fetch account balance by ID",
        "parameters": {"type": "object", "properties": {"account_id": {"type": "string"}}, "required":
["account_id"]}}
}]
resp = client.chat.completions.create(model="gpt-4o", messages=messages, tools=tools)
tool_call = resp.choices[0].message.tool_calls[0]
if tool_call.function.name == "get_account_balance":
    args = json.loads(tool_call.function.arguments)
    result = get_account_balance(args["account_id"])   # your real function — validate args first!
    messages.append({"role": "tool", "tool_call_id": tool_call.id, "content": str(result)})
    final = client.chat.completions.create(model="gpt-4o", messages=messages)
```

PREP Interview Q&A

Q: How do you keep an autonomous agent from taking a harmful or costly action? SENIOR

I put human-in-the-loop approval gates in front of anything irreversible or high-risk, keep a strict allow-list of tools per agent based on least privilege, validate input and output on every tool call, and set hard caps on iterations and spend. Basically I treat agent tool access exactly like an API permission model rather than a suggestion, with full audit logging on top.

Q: Difference between a chatbot and an agent? CORE

A chatbot maps input to output text in one shot, maybe with retrieval added in. An agent runs an actual reasoning loop where it can decide to call tools, look at real-world results, and adjust its plan across several steps before it gives you a final answer. The real difference is autonomy over the path to the answer, not the answer itself.

TOPIC 11 • AGENTIC AI

Multi-Agent Systems

When one agent isn't enough — orchestrator/worker patterns used in real production systems.

KEY POINTS Common Patterns

Pattern	How It Works	Real Example
Orchestrator-Worker	A supervisor agent decomposes a task and routes sub-tasks to specialist agents	Loan-processing supervisor routes to Document-Extraction, Risk-Scoring, and Compliance agents
Hierarchical	Manager agents oversee teams of worker agents, each with narrow tools	Research manager + web-search agent + summarizer agent
Debate/Critique	Two agents argue or one critiques the other's output before finalizing	Generator agent + Fact-checker agent loop before publishing
Sequential Pipeline	Fixed hand-off order, each agent's output is the next's input	Extract → Validate → Summarize → Notify

TIPS Production Reality

Shared state/memory design | **Inter-agent message format** | **Failure isolation per agent** | **Cost multiplies per agent hop** | **Observability across the whole trace** | **Deadlock/loop guards**

PREP Interview Q&A

Q: Design a multi-agent system for automated loan document processing. SENIOR

I'd set up a Supervisor agent that receives the application and plans the steps, an Extraction agent with OCR and parsing tools to pull structured fields from uploaded PDFs, a Validation agent to cross-check that data against business rules, a Risk-Scoring agent that calls an internal ML model, and a Compliance agent for regulatory flags. The Supervisor pulls it all together and either auto-approves, escalates to a human, or asks for missing documents. Each agent gets a narrow tool scope, and every hand-off is logged for audit.

Q: What's the biggest risk of multi-agent systems in production and how do you manage it? SENIOR

Cost and latency both compound with every agent hop, and if agents keep re-delegating to each other, failures can cascade or loop indefinitely. I manage that with a global step budget enforced at the orchestrator level, circuit breakers on each agent, and a fallback to a simpler single-agent path if the multi-agent flow ever times out.

PRODUCTION Real-World Example & Python Interview Q&A

Real-world example: I once watched a multi-agent research pipeline get stuck re-delegating the exact same sub-task back and forth between two agents, basically an infinite ping-pong. The fix was adding a shared `visited_steps` set to state, so any agent trying to re-delegate a sub-task it had already handled in that run got rejected outright.

Q: In Python, how would you implement a simple message bus for agents to communicate asynchronously? SENIOR

My go-to is asyncio.Queue per agent, or a shared queue with routing keys. Each agent runs as an async task consuming from its own queue and publishing results to the others, and I wrap the whole thing in asyncio.gather with a timeout so one stuck agent can't block the entire run. It mirrors how production multi-agent frameworks handle message passing under the hood.

TOPIC 12 • QUALITY & SAFETY

Model Evaluation & Guardrails

How you prove an LLM system is production-ready — and keep it that way.

CONCEPT Key Points

- **Offline eval**: golden Q&A sets, faithfulness/groundedness scoring, LLM-as-judge, exact/semantic match for extraction tasks.
- **Online eval**: thumbs up/down, implicit signals (re-asks, escalation to human), A/B testing new prompts/models.
- **Guardrails**: input filters (PII, toxicity, injection), output filters (schema validation, banned-topic checks), rate/action limits.

Metric to know: *Faithfulness* = does the answer only contain claims supported by retrieved context? Distinct from *Answer Relevance* = does it actually address the question?

Tooling: RAGAS, DeepEval, TruLens, LangSmith evals, Azure AI Content Safety, Guardrails AI / NeMo Guardrails for policy enforcement.

CODE Code: Simple LLM-as-Judge Eval

```
JUDGE_PROMPT = (
    "Rate the ANSWER's faithfulness to CONTEXT on 0-1. Return only a number.\n"
    "If ANSWER states anything not supported by CONTEXT, score it low.\n"
    "CONTEXT: {context}\nANSWER: {answer}"
)

def faithfulness_score(context, answer) -> float:
    resp = client.chat.completions.create(
        model="gpt-4o-mini", temperature=0,
        messages=[{"role": "user", "content": JUDGE_PROMPT.format(context=context, answer=answer)}]
    )
    return float(resp.choices[0].message.content.strip())

# Run across a golden set and gate deployment on average score
scores = [faithfulness_score(c, a) for c, a in golden_set]
assert sum(scores)/len(scores) > 0.85, "Faithfulness below release threshold"
```

PREP Interview Q&A

Q: How do you evaluate an LLM system without ground-truth labels for every query? SENIOR

I build a golden set first, usually 100 to 300 representative labeled queries. LLM-as-judge handles scale on the rest of unlabeled traffic, with periodic human spot-checks to keep the judge honest, and I track proxy signals like thumbs-down rate, escalation rate, and re-ask rate as an early warning between full evals.

Q: A guardrail is blocking too many legitimate customer messages — how do you fix it? SENIOR

I pull the false-positive logs first and cluster them to find the actual pattern, which is usually an overly broad PII or toxicity regex. From there I either tighten the rule or replace a keyword-based filter with a calibrated classifier, run the previously blocked set through the updated guardrail before redeploying, and A/B the change against the block/allow rate rather than just eyeballing it.

PRODUCTION Real-World Example & Python Interview Q&A

Real-world example: I had a faithfulness eval pass cleanly in CI, and the model still hallucinated in production on a rare document type. Turned out our golden set had zero examples of that document type in it. Adding a stratified sampling step so every document category actually shows up in the eval set solved it for good.

Q: Write a Python decorator that retries a flaky LLM API call with exponential backoff. SENIOR

I'd write it using functools.wraps, catch the target exception, sleep for base_delay times 2 to the power of attempt (with some jitter thrown in), and re-raise once max_retries is hit. It's a pattern I put on every external LLM or API call in production, since transient 429s and 500s happen often enough that they shouldn't be allowed to crash the whole request.

TOPIC 13 • PRODUCTION

LLMOps / MLOps for LLMs

Deploying, monitoring, and iterating on LLM systems like real software.

CONCEPT Key Points

- **CI/CD for prompts**: version-control prompts like code, run eval suite on every change, gate deploys on score thresholds.
- **Observability**: trace every call (prompt, response, latency, token cost, model version) — LangSmith, Arize Phoenix, Datadog LLM Observability.
- **Model versioning**: pin model versions explicitly; provider silent upgrades can shift behavior overnight.
- **Canary/shadow deploys**: route a small % of traffic to a new prompt/model version and compare metrics before full rollout.
- **Cost monitoring**: track $/request and $/user; set budget alerts — token cost is the new cloud-bill surprise.

TROUBLESHOOTING Production Troubleshooting: Common Incidents

Symptom	Likely Cause	Fix
Sudden quality drop after no code change	Provider silently updated default model version	Pin exact model version string; add regression eval to CI
Latency spike (p99)	No streaming, cold starts, or provider-side throttling	Enable streaming, add retries w/ backoff, autoscale warm pool
Cost spike	Agent loop not capped, or context growing unbounded	Cap iterations, truncate/summarize long histories, cache repeat calls
Inconsistent output format	Free-text generation instead of structured output	Use function calling / JSON schema mode + output validation

PREP Interview Q&A

Q: Describe your LLMOps pipeline from prompt change to production. SENIOR

Any prompt or config change goes through version control first, which kicks off an automated eval run against the golden set and regression suite. It has to hit the faithfulness or accuracy threshold before it's allowed anywhere near production, then it deploys to a canary slice, usually 5 to 10 percent of traffic, with live monitoring on latency, cost, and quality. If the canary metrics hold for a defined window it auto-promotes to full traffic, and rollback is wired into that same CI pipeline so it's instant if something goes wrong.

PRODUCTION Real-World Example & Python Interview Q&A

Real-world example: A canary deploy I worked on looked completely healthy on average latency, but one specific tenant's traffic had silently regressed underneath that average. After that, we started slicing monitoring by tenant and segment instead of only looking at the global number. Aggregate metrics hide exactly the outliers that matter most.

Q: How would you implement async batching of LLM calls in Python to improve throughput? SENIOR

asyncio.gather paired with a semaphore is my usual approach, capping how many requests are in flight at once to respect the provider's rate limit. Each call goes through the retry-with-backoff decorator, and if I need results as they arrive rather than waiting on the whole batch, asyncio.as_completed handles that. It's fairly standard for any high-throughput LLMOps pipeline.

TOPIC 14 • AWS

AWS SageMaker for LLMs

Training, fine-tuning, and serving LLMs on AWS.

CONCEPT Key Points

- **SageMaker JumpStart**: one-click deploy of foundation models (Llama, Mistral) to a managed endpoint.
- **SageMaker Training Jobs**: managed, scalable fine-tuning (LoRA/QLoRA) with spot instance cost savings.
- **SageMaker Endpoints**: real-time inference (autoscaling) or async/batch inference for high-volume, latency-tolerant workloads.
- **Bedrock** (adjacent service): fully managed foundation-model API — no infra to manage, use when you don't need custom training infra.

When to use SageMaker vs Bedrock: Bedrock for fast integration of off-the-shelf foundation models via API; SageMaker when you need custom fine-tuning infrastructure, specific GPU instance control, or a model not available on Bedrock.

CODE Code: Deploy a Fine-tuned Model Endpoint

```
from sagemaker.huggingface import HuggingFaceModel

hf_model = HuggingFaceModel(
    model_data="s3://my-bucket/lora-finetuned-model.tar.gz",
    role=sagemaker_role,
    transformers_version="4.37", pytorch_version="2.1", py_version="py310"
)

# Deploy to a real-time autoscaling endpoint
predictor = hf_model.deploy(
    initial_instance_count=1, instance_type="ml.g5.2xlarge",   # GPU instance for LLM inference
    endpoint_name="policy-extraction-endpoint"
)
result = predictor.predict({"inputs": "Extract key clauses from: ..."})
```

PREP Interview Q&A

Q: Your SageMaker LLM endpoint is expensive and underutilized overnight — how do you optimize cost? SENIOR

I'd move off a provisioned real-time endpoint and onto SageMaker Serverless or Asynchronous Inference for that bursty, overnight-light traffic pattern. Autoscaling with a scale-to-zero config where SLAs allow it, plus Savings Plans or Spot on the training side, so the instance type and lifecycle actually match how traffic behaves instead of provisioning for peak load 24/7.

PRODUCTION Real-World Example & Python Interview Q&A

Real-world example: An endpoint I was responsible for OOM-crashed during a burst of unusually long-context requests. The lasting fix was a request-size guard inside the inference handler itself, written in Python, that rejects or chunks oversized payloads before they ever reach the model, plus CloudWatch alarms watching GPU memory utilization going forward.

Q: Write a Python inference handler function for a SageMaker custom endpoint. SENIOR

A SageMaker HuggingFace handler typically overrides four functions: model_fn to load the model once at container startup, input_fn to parse the incoming payload, predict_fn to actually run generate(), and output_fn to serialize the response. I make sure model_fn's load happens outside the request path since it's expensive, and I validate input_fn's payload size before it ever reaches predict_fn so bursty traffic can't OOM-crash the endpoint.

TOPIC 15 • GCP

GCP Vertex AI for LLMs

Google Cloud's managed platform for Gemini, PaLM, and custom model deployment.

CONCEPT Key Points

- **Vertex AI Model Garden**: catalog of foundation models (Gemini, open-source) deployable with managed endpoints.
- **Vertex AI Pipelines**: Kubeflow-based orchestration for repeatable training/eval/deploy workflows.
- **Vertex AI Search / Grounding**: managed RAG — built-in retrieval grounding for Gemini without building your own vector pipeline.
- **Vertex AI Feature Store & Model Monitoring**: drift detection and feature consistency between training and serving.

CODE Code: Gemini via Vertex AI SDK

```
import vertexai
from vertexai.generative_models import GenerativeModel, GenerationConfig

vertexai.init(project="my-gcp-project", location="us-central1")
model = GenerativeModel("gemini-1.5-pro")

config = GenerationConfig(temperature=0.2, max_output_tokens=1024, response_mime_type="application/json")
response = model.generate_content(
    "Summarize this transaction dispute and classify severity.",
    generation_config=config
)
print(response.text)   # structured JSON, ready to parse
```

PREP Interview Q&A

Q: How would you set up automated retraining/monitoring for a production model on Vertex AI? SENIOR

Vertex AI Model Monitoring tracks prediction and feature drift against the training baseline, and I'd wire alerts from that straight into a Vertex AI Pipeline that retrains and re-evaluates on fresh data. Promotion only happens once the eval step clears its threshold, and the new model version gets registered in Vertex Model Registry with full lineage before any production traffic touches it.

PRODUCTION Real-World Example & Python Interview Q&A

Real-world example: A Vertex AI Pipeline step was silently failing on about 1% of documents that had unusual encoding, and nobody noticed for a while because the job itself reported success. Wrapping each pipeline component's Python code in explicit try/except blocks with structured logging per document ID, instead of letting the whole job fail generically, is what surfaced it.

Q: How do you handle a mix of sync and async code when integrating Vertex AI calls into an existing Python service? SENIOR

I keep the async Vertex AI SDK calls behind a small async wrapper module. If the surrounding service is synchronous, say Flask, I run that async call through asyncio.run() inside a dedicated thread pool executor rather than blocking the main request thread. For a fully async service like FastAPI, I just await it directly, which keeps I/O-bound model calls from starving everything else.

TOPIC 16 • EFFICIENCY

Scaling & Cost Optimization

Techniques that separate a prototype from a production GenAI system at scale.

CONCEPT Key Techniques

Technique	What It Does	Impact
Prompt/response caching	Store results for repeated/similar queries	Cuts redundant API cost + latency to near-zero on cache hit
Batching	Group multiple requests into one inference call	Higher GPU utilization, lower cost per request
Quantization (INT8/ INT4)	Reduce model weight precision	2-4x smaller memory footprint, faster inference, small quality tradeoff
KV-cache reuse	Cache attention key/values across generation steps	Avoids recomputation, standard in all production servers (vLLM, TGI)
Model routing / cascading	Send easy queries to a cheap small model, hard ones to a large model	30-70% cost reduction with minimal quality loss
Streaming	Return tokens as generated instead of waiting for full response	Massively improves perceived latency (TTFT)

TIPS Serving Stack

vLLM | TGI (Text Generation Inference) | Triton Inference Server | Ray Serve | LMDeploy

PREP Interview Q&A

Q: Your GenAI feature's API bill tripled in a month with flat user growth — walk me through your investigation. SENIOR

I start by pulling per-endpoint token usage to find where the cost is actually coming from. From there I check for a recent prompt change that bloated context, like unsummarized chat history creeping in, look for retry storms from a flaky downstream call, check whether agent loops are hitting max iterations more than usual, and rule out a model version quietly switching to a pricier tier. Once I find the root cause, I add a cost-per-request dashboard with alerting so it gets caught in days next time, not a month.

Q: When would model routing (small model + large model cascade) hurt more than help? SENIOR

The moment the classifier deciding 'easy vs hard' isn't reliable. Misrouting complex queries to the small model causes quality to degrade quietly, and that's actually harder to catch than a flat cost increase would be. I only adopt cascading after validating the router's accuracy on a labeled, difficulty-graded eval set, and I keep monitoring quality metrics split by routing decision after that.

PRODUCTION Real-World Example & Python Interview Q&A

Real-world example: I dropped a simple LRU cache in front of an embedding call and it cut a document-processing job's runtime by 40%, mostly because so many chunks across similar documents turned out to be near-duplicates. Caching by a content hash of the chunk text meant we stopped re-embedding the same thing over and over.

Q: How would you implement a simple in-memory LRU cache for LLM responses in Python? SENIOR

functools.lru_cache works fine for a quick single-process cache, keyed on a hash of the prompt, model, and temperature together. For production I'd move that to a Redis-backed cache with a TTL so it survives restarts and is shared across worker processes. The real design decision is picking a cache key that captures everything that affects the output, otherwise you end up serving stale or just wrong responses.

TOPIC 17 • OPERATIONS

Production Troubleshooting Playbook

The on-call runbook — what breaks in LLM systems and how you fix it fast.

TROUBLESHOOTING Incident Playbook

Issue	Root Cause Checklist	Resolution
Response truncated mid-answer	max_tokens too low; context exceeded model limit	Raise max_tokens, trim/summarize history, chunk long outputs
Hallucinated facts in RAG answer	Retrieval missed the right chunk; weak grounding instruction	Improve chunking/embedding, add citation-forcing prompt, lower temperature
Rate limit (429) errors under load	No backoff/retry; burst traffic exceeds quota	Exponential backoff + jitter, request queueing, quota increase request
PII leaking into logs/output	No input/output redaction layer	Add PII scrubber (Presidio) pre-log and pre-response
Agent stuck in infinite tool-call loop	No max-iteration cap; ambiguous stop condition	Hard cap iterations, add explicit termination criteria in prompt
Embedding/query model mismatch	Index built with model A, querying with model B	Always pin and version-match embedding model between ingest and query

TIPS Debugging Approach (Interview-Ready)

- **1. Isolate the layer** — retrieval, prompt, model, or post-processing? Test each independently.
- **2. Reproduce with a fixed seed/temperature=0** to rule out sampling randomness.
- **3. Check traces** (LangSmith/Phoenix) for the exact prompt sent and raw model output before your parser touched it.
- **4. Compare against the golden eval set** — did this fail before, or is it a regression?

PREP Interview Q&A

Q: Walk me through debugging a production incident where a GenAI feature started returning empty responses for 5% of users. SENIOR

I'd look for a correlation first, whether it's long inputs hitting the context limit, a specific language, or special characters breaking the prompt template. Then I'd check provider status and error codes in the logs, since content filter blocks return empty completions rather than actual errors. I'd also verify the output parser isn't silently swallowing malformed JSON, and add a fallback response plus alerting so this kind of failure never shows up as silence to the user again.

PYTHON Python Interview Q&A for Troubleshooting

Q: Write a Python context manager that logs the latency of an LLM call. SENIOR

I'd implement __enter__ and __exit__, or just use contextlib.contextmanager with a try/finally, to grab time.perf_counter() before and after the wrapped block. Log the elapsed time along with the request ID, and make sure __exit__ re-raises any exception so the caller still sees failures. This is something I wrap around basically every external call in a codebase that takes observability seriously.

Q: How do you debug a Python memory leak in a long-running LLM inference service? SENIOR

tracemalloc or memory_profiler to snapshot memory over time is usually my first move. From there I'm looking for unbounded caches or lists that keep growing, like conversation history that's never pruned, checking that large tensors and embeddings actually get released (del plus gc.collect() around GPU tensors if it's PyTorch), and confirming the connection pool to the vector DB isn't leaking sockets on every request.

TOPIC 18 • TOOLBOX

Python Libraries Reference

The stack you'll actually touch day to day, and where each one fits.

REFERENCE Core Libraries & Production Use

Library	Purpose	Production Usage
openai / anthropic SDKs	Model API calls	Chat completions, function calling, streaming, embeddings
langchain / langgraph	Orchestration	RAG chains, agent state machines, memory, tool routing
llama-index	Data framework for RAG	Advanced indexing (tree, keyword, graph), query engines
transformers (HF)	Model loading/fine-tuning	Local inference, tokenizers, Trainer API for fine-tuning
peft / trl	Efficient fine-tuning	LoRA/QLoRA fine-tuning, RLHF/DPO training loops
sentence-transformers	Embeddings	Local/open-source embedding generation for RAG
pinecone-client / chromadb / faiss	Vector storage	Index, upsert, ANN search for retrieval
pydantic	Data validation	Structured LLM output parsing/validation before downstream use
tiktoken	Tokenization	Token counting for cost/context budgeting
vllm	High-throughput serving	Self-hosted LLM inference with PagedAttention, batching
ragas / deepeval / trulens	RAG/LLM evaluation	Faithfulness, relevance, groundedness scoring in CI
fastapi	API layer	Wrapping LLM pipelines as production microservices

CODE Code: Minimal Production Service Skeleton

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Query(BaseModel):
    question: str

class Answer(BaseModel):
    answer: str; citations: list[str]; confidence: float

@app.post("/ask", response_model=Answer)
async def ask(q: Query):
    chunks = retriever.get_relevant_documents(q.question)          # 1. retrieve
    result = await chain.ainvoke({"question": q.question, "context": chunks})  # 2. generate
    return Answer(**result)                                                    # 3. validated response
```

PYTHON Python Interview Questions for AI/ML Engineers

Q: What's the GIL and why does it matter when serving LLM requests in Python? SENIOR

The Global Interpreter Lock means only one thread executes Python bytecode at a time, so CPU-bound multithreading doesn't actually parallelize. LLM API calls are I/O-bound though, just waiting on the network, so asyncio or even plain threading still gives you real concurrency there. It's only for genuinely CPU-bound work, like local tokenization or embedding at scale, that I'd reach for multiprocessing to get around the GIL.

Q: How do you validate and enforce a strict schema on LLM JSON output in Python? SENIOR

I define a Pydantic model matching whatever schema I expect and call Model.model_validate_json() on the raw response. If that raises a ValidationError, I either retry with a corrective prompt or route to a fallback rather than letting malformed data flow downstream unnoticed. It's probably the single highest-leverage Python pattern for getting reliable output out of an LLM.

TOPIC 19 • INTERVIEW PREP

Company-Specific Interview Styles (1/2)

How Meta, Google, Amazon, and Microsoft actually run GenAI/ML engineering interviews.

KEY POINTS What Changes By Company

Meta (AI/GenAI Engineer)

Focus: Applied ML depth, system design at massive scale, PyTorch internals
Interview style: Heavy coding bar (LeetCode medium/hard) + ML system design (e.g. design a ranking/recommendation or GenAI feature for billions of users) + behavioral on 'Move Fast'.

Sample prompt: *"Design a system to generate personalized ad copy for 2 billion users with sub-200ms latency and safety filtering."*

Google (ML/GenAI Engineer, Vertex/DeepMind-adjacent)

Focus: Algorithmic rigor, distributed systems, research-grounded reasoning
Interview style: GCA (Googleyness) + coding + ML system design with strong emphasis on 'why', trade-off reasoning, and first-principles math (attention complexity, scaling laws).

Sample prompt: *"How would you reduce serving cost for a Gemini-based feature by 50% without materially hurting quality?"*

Amazon (Applied Scientist / GenAI Engineer, Bedrock/AWS)

Focus: Leadership Principles woven into every answer, practical AWS-native architecture
Interview style: STAR-format behavioral tied to Leadership Principles (Ownership, Bias for Action, Dive Deep) interleaved with technical rounds; system design expects AWS service fluency (Bedrock, SageMaker, OpenSearch).

Sample prompt: *"Tell me about a time you dove deep into a production issue no one else could solve. Then: design a RAG system on AWS for 10M documents."*

Microsoft (AI/Copilot Engineer, Azure OpenAI)

Focus: Enterprise integration, Azure ecosystem, responsible AI
Interview style: Collaborative, less adversarial coding rounds; strong focus on Azure OpenAI Service, Semantic Kernel/AutoGen, and Microsoft's Responsible AI framework; growth-mindset behavioral questions.

Sample prompt: *"How would you add a GenAI Copilot feature to an enterprise product while meeting Microsoft's Responsible AI guidelines?"*

TIPS Universal Prep Tip

Across all four, they're really listening for the same things: can you reason about trade-offs out loud (cost vs latency vs accuracy), can you ground your answer in a real project you actually shipped, and do you understand both the ML and the production system around it (data pipeline, serving, monitoring), not just the model theory on its own.

PYTHON Python Interview Angle at These Companies

Q: Meta/Google style: optimize this Python function that re-embeds every chunk on every request. SENIOR

First thing I'd flag is the O(n) redundant recomputation. I'd add memoization keyed on a content hash, using functools.lru_cache or a Redis cache if it needs to persist across processes, and if it's still a hot path after that, move to batch embedding calls instead of one at a time. The goal is showing I can reason about algorithmic complexity, not just call an SDK and move on.

Q: Amazon/Microsoft style: how do you write testable Python code for an LLM pipeline? SENIOR

I put the non-deterministic model call behind a thin interface, essentially dependency injection, so tests can mock it out. The deterministic parts, prompt building, output parsing, business logic, get unit-tested in isolation, and I keep a small number of real-model integration tests with a fixed seed and temperature=0 for CI. That's the level of engineering rigor Amazon and Microsoft interviews tend to probe for specifically.

TOPIC 19 • INTERVIEW PREP

Company-Specific Interview Styles (2/2)

Tesla, Apple, and the Banking/Fintech interview loop.

KEY POINTS What Changes By Company

Tesla (AI/ML Engineer)

Focus: Speed, first-principles thinking, real-time/edge constraints
Interview style: Fast-paced, expects you to justify every design decision from first principles; less emphasis on frameworks, more on 'could you have built this yourself'; strong bar on production/real-time systems thinking.

Sample prompt: *"Design an LLM-based assistant for in-vehicle use with strict latency and offline-fallback requirements."*

Apple (ML/AI Engineer)

Focus: Privacy-first design, on-device inference, cross-functional polish
Interview style: Emphasis on privacy-preserving architecture (on-device vs cloud), tight collaboration with product/design, and quality bar on user-facing polish; interviews often probe how you'd keep user data off servers.

Sample prompt: *"Design a GenAI writing-assistance feature that never sends user text to the cloud."*

Banking / Fintech (matches your Infosys background)

Focus: Regulatory compliance, auditability, risk-aversion, data privacy
Interview style: Expect deep questions on explainability, PII handling, model governance/audit trails, and 'what happens when the model is wrong' — regulators require traceability that pure tech companies don't.

Sample prompt: *"How do you ensure a GenAI document-QA assistant used by compliance officers is auditable and never leaks customer PII?"*

Your edge: You already have banking-domain war stories (Azure ADF pipelines, real client environments). Lead with those — regulated-industry GenAI experience is rare and highly valued at every one of these companies.

PREP Interview Q&A — Cross-Company Favorite

Q: Tell me about the most technically challenging GenAI/ LLM system you've built. SENIOR

I structure it the same way every time: the business problem, why a naive prompt-only approach didn't hold up, the architecture I actually landed on and the RAG-versus-agents-versus-fine-tuning trade-off behind it, one specific production issue I hit and how I diagnosed and fixed it, and the measurable outcome, whether that's latency, cost, or accuracy. A real failure-then-fix story with concrete numbers beats a clean success story every single time.

Q: How do you stay current as this field moves so fast? CORE

I stick to primary sources, provider release notes, arXiv for architecture shifts, rather than secondhand takes on social media. When a technique looks genuinely production-relevant I'll rebuild a small prototype and test it on a real dataset before I trust it, and I try to validate hype against my own eval numbers instead of adopting something just because it's trending.

PYTHON Python Interview Q&A

Q: Tesla/Apple style: how would you write Python code that must run on constrained edge hardware? SENIOR

I profile memory and CPU before I optimize anything, since guessing wastes time. I lean toward generators and iterators instead of building large lists in memory, avoid pulling in full frameworks for something one function could handle, and rely on quantized or compiled models like ONNX Runtime or Core ML invoked from a thin Python or native layer rather than doing heavy compute in pure Python.

Q: Banking style: how do you make Python code auditable for a compliance review? SENIOR

Business logic lives in pure, well-named functions with type hints and docstrings. Every decision point gets structured logging instead of print statements, I avoid mutable global state, and I pin all dependency versions in requirements.txt or poetry.lock so an auditor can reproduce exact behavior from a given commit. Reproducibility is really what the compliance team is asking for, even when they phrase it as 'explain the code.'

PRACTICE PROJECT 1 OF 5 • META

Personalized Ad Copy Generator at Scale

A GenAI system that generates thousands of personalized ad-copy variants per campaign, optimized for engagement while enforcing brand-safety and latency budgets at billions-of-requests scale.

FLOW Architecture

- **Step 1:** A ranking/targeting service selects the audience segment and product context for each ad slot.
- **Step 2:** A prompt template merges product data with audience signals. A smaller distilled model, routed through a cascade, drafts copy for low-risk segments, and a larger model takes the high-value or brand-sensitive ones.
- **Step 3:** A safety/guardrail classifier screens every variant for policy violations before it enters the ad auction.
- **Step 4:** Generated variants get cached and reused across similar audience segments to cut inference cost at scale.
- **Step 5:** Online A/B testing continuously measures engagement lift and feeds that back into which model or prompt variant wins.

TOOLS Tech Stack

PyTorch | Model Distillation | Redis Cache | Content Safety Classifier | A/B Testing Pipeline

CODE Starter Code

```
# Cascade: cheap model for most traffic, escalate only when confidence is low
def generate_ad_copy(product, audience):
    draft = small_model.generate(product, audience)
    if safety_classifier.score(draft) < THRESHOLD or audience.is_high_value:
        draft = large_model.generate(product, audience)   # escalate only when needed
    return draft if safety_classifier.score(draft) >= THRESHOLD else FALLBACK_COPY
```

PREP Interview Angle

Q: At billions-of-requests scale, how do you keep GenAI ad-copy generation both cheap and safe? SENIOR

The trick is a model cascade. A small distilled model handles most of the low-risk traffic, and only ambiguous or high-value segments get escalated to a larger model. Add caching for near-duplicate audience segments and a hard safety-classifier gate before anything reaches the auction, and cost grows much slower than traffic while brand-safety risk stays bounded.

Why this lands well at Meta: It shows you can reason about system design at Meta's actual scale, weigh cost against latency against safety out loud, and speak PyTorch fluently, which is exactly what their ML system design round is probing for.

PYTHON Python Interview Question

Q: Write Python logic to deduplicate near-identical ad-copy requests before hitting the model. SENIOR

Normalize the (product, audience_segment) pair into a key, hash it, and check a Redis or LRU cache before calling the model at all. A cache hit returns instantly, and a short TTL keeps stale copy from running forever. At Meta's request volume, this one pattern alone is often the biggest lever on cost.

PRACTICE PROJECT 2 OF 5 • GOOGLE

Multimodal Search Grounding Assistant

A Gemini-powered assistant that answers questions using both text documents and images/screenshots, grounded via Vertex AI Search, for an internal knowledge-base product.

FLOW Architecture

- **Step 1:** Documents, both text and images, are ingested into Vertex AI Search, which builds a managed hybrid (keyword plus semantic) index.
- **Step 2:** A user query, whether text or image, goes to a Gemini multimodal model with Vertex AI Search grounding turned on.
- **Step 3:** The model retrieves and cites the most relevant passages or images automatically through the grounding API.
- **Step 4:** A confidence and attribution check makes sure every claim in the answer traces back to a retrieved source before it's returned.
- **Step 5:** Vertex AI Model Monitoring keeps an eye on answer quality and retrieval drift over time.

TOOLS Tech Stack

Vertex AI Search | Gemini 1.5 Pro (multimodal) | Vertex AI Model Monitoring | Cloud Storage | BigQuery (analytics)

CODE Starter Code

```
from vertexai.generative_models import GenerativeModel, Tool, grounding

model = GenerativeModel("gemini-1.5-pro")
search_tool = Tool.from_retrieval(
    grounding.Retrieval(grounding.VertexAISearch(datastore="projects/.../dataStores/kb"))
)
response = model.generate_content(
    ["What does the Q3 architecture diagram show about the ingestion path?", image_part],
    tools=[search_tool]   # grounding: answer must cite retrieved sources
)
```

PREP Interview Angle

Q: Why use Vertex AI Search grounding instead of building your own RAG pipeline? SENIOR

On a Google-native stack, Vertex AI Search already gives you managed hybrid retrieval, automatic citations, and freshness handling, so I don't have to maintain chunking, embedding, and indexing infrastructure myself. I'd only reach for a custom RAG pipeline if I needed retrieval logic it doesn't support, like a very specific re-ranking model or a data residency requirement outside GCP.

Why this lands well at Google: It shows real fluency with their production stack, Vertex AI Search plus Gemini multimodal, and multimodal grounding specifically is something Google interviewers like to probe, since it goes beyond the usual text-only RAG answer.

PYTHON Python Interview Question

Q: How would you paginate through and process millions of documents for ingestion in Python without running out of memory? SENIOR

Use a generator-based ingestion loop that pulls one batch at a time with cursor-based pagination rather than offset-based, which degrades badly at scale. Process and embed each batch, then discard it before pulling the next one, so you never hold the full document set in memory at once.

PRACTICE PROJECT 3 OF 5 • MICROSOFT

Enterprise Copilot with Semantic Kernel

A Microsoft 365-style Copilot plugin that lets employees ask natural-language questions across SharePoint documents and Teams messages, built on Azure OpenAI Service with Responsible AI controls.

FLOW Architecture

- **Step 1:** Azure OpenAI Service hosts the base model (GPT-4o) behind Azure's enterprise compliance boundary.
- **Step 2:** Semantic Kernel orchestrates a set of 'skills,' or plugins: one for SharePoint search, one for Teams message search, one for calendar lookup.
- **Step 3:** The planner, Semantic Kernel's function-calling planner, decides which skill or skills to invoke for a given query.
- **Step 4:** Azure AI Content Safety screens both the input and the output for policy compliance before the response ever reaches the user.
- **Step 5:** Azure Monitor and Application Insights track latency, token cost, and safety-block rate per tenant.

TOOLS Tech Stack

Azure OpenAI Service | Semantic Kernel | Azure AI Content Safety | Microsoft Graph API | Application Insights

CODE Starter Code

```
from semantic_kernel import Kernel
from semantic_kernel.connectors.ai.open_ai import AzureChatCompletion

kernel = Kernel()
kernel.add_service(AzureChatCompletion(deployment_name="gpt-4o", endpoint=AZURE_ENDPOINT))
kernel.add_plugin(SharePointSearchPlugin(), plugin_name="SharePoint")
kernel.add_plugin(TeamsSearchPlugin(), plugin_name="Teams")

# Planner auto-selects which plugin(s) to call based on the user's question
result = await kernel.invoke_prompt("What did the finance team decide about Q3 budget in last week's Teams
thread?")
```

PREP Interview Angle

Q: How do you keep a multi-source enterprise Copilot compliant with Microsoft's Responsible AI standards? SENIOR

Every skill call is scoped to the requesting user's actual Microsoft Graph permissions, so there's no privilege escalation through the model. All input and output goes through Azure AI Content Safety, every response is logged with full plugin-call lineage for audit, and any new plugin gets a Responsible AI impact assessment before it's added to the planner's toolset.

Why this lands well at Microsoft: It's built on their actual enterprise stack, Semantic Kernel, Azure OpenAI, Graph API, and it puts Responsible AI at the center rather than as an afterthought, which is exactly the framing their Copilot engineering interviews reward.

PYTHON Python Interview Question

Q: How would you enforce per-user permission scoping in Python when a plugin queries SharePoint on behalf of a user? SENIOR

Never let the plugin use a service-account token with broad access. Pass the requesting user's delegated OAuth token through to the Graph API call instead, so results are automatically filtered to what that person can actually see, and add an explicit assertion in code that blocks any plugin call missing a user-scoped token.

PRACTICE PROJECT 4 OF 5 • TESLA

In-Vehicle Voice Assistant with Offline Fallback

A voice assistant for in-car use that must respond in real time, work with degraded or no connectivity, and never let a cloud outage break basic vehicle controls.

FLOW Architecture

- **Step 1:** A small on-device (edge) quantized LLM handles core commands like climate, navigation shortcuts, and media control with zero network dependency.
- **Step 2:** Complex or open-ended queries, like trip planning or general Q&A, only route to a cloud LLM once connectivity is confirmed.
- **Step 3:** A local intent classifier decides on-device versus cloud routing in under 50ms so there's no perceptible lag.
- **Step 4:** If the cloud call times out or fails, the system falls back to a scripted response instead of hanging or throwing an error.
- **Step 5:** All voice processing for core vehicle commands stays on-device, both for latency and for privacy.

TOOLS Tech Stack

Quantized On-Device LLM (INT4/INT8) | ONNX Runtime | Edge Inference | Intent Classifier | Graceful Degradation Logic

CODE Starter Code

```
def handle_voice_command(audio):
    intent = local_intent_classifier(audio)   # <50ms, always on-device
    if intent.category == "core_vehicle_control":
        return onboard_llm.execute(intent)          # never depends on network
    if network.is_available():
        try:
            return cloud_llm.generate(intent, timeout=1.5)
        except TimeoutError:
            pass
    return SCRIPTED_FALLBACK[intent.category]   # never leave the driver with silence
```

PREP Interview Angle

Q: Why not just always route voice commands to a large cloud LLM for the best answer quality? SENIOR

Because a moving vehicle can't treat latency or connectivity as something it can always count on. I keep safety- and control-critical commands fully on-device with a small quantized model, save the cloud model for genuinely open-ended queries where a second of delay is fine, and always build an explicit fallback so a network hiccup degrades gracefully instead of breaking the assistant outright.

Why this lands well at Tesla: It speaks directly to their real-time and edge-constraint bar. You're designing around hardware and connectivity limits, not just model quality, and that's the kind of first-principles thinking their system design interviews are built to surface.

PYTHON Python Interview Question

Q: How would you enforce a hard timeout on a Python function call to a cloud model so it never blocks the vehicle's UI thread? SENIOR

asyncio.wait_for around the coroutine with an explicit timeout works well, or concurrent.futures with a timeout on the future's result() call if the code is synchronous. Either way, on timeout you catch the exception and immediately return the scripted fallback instead of leaving the caller waiting.

PRACTICE PROJECT 5 OF 5 • APPLE

On-Device Privacy-First Writing Assistant

A writing-assistance feature (tone rewrite, summarization, proofreading) that runs entirely on-device so user text never leaves the phone, matching Apple's privacy-by-design approach.

FLOW Architecture

- **Step 1:** A compact on-device LLM, quantized and distilled specifically for the writing-assistance task, ships bundled with the OS or app.
- **Step 2:** All standard text processing (rewrite, summarize, proofread) happens locally. No network call is made for a normal request.
- **Step 3:** For advanced requests beyond the on-device model's capability, an explicit, user-visible consent step is required before anything reaches a cloud service, and even then only an anonymized snippet goes out.
- **Step 4:** Differential privacy techniques get applied to any aggregate model-improvement telemetry that's collected.
- **Step 5:** The feature plugs directly into system text fields through an on-device inference API instead of a network-backed service.

TOOLS Tech Stack

Core ML | On-Device Quantized LLM | Differential Privacy | Distillation | Local Inference API

CODE Starter Code

```
def rewrite_text(text, tone):
    # Fully local inference - no network call, no server ever sees raw text
    result = on_device_model.generate(prompt=build_prompt(text, tone), max_tokens=256)
    return result   # nothing leaves the device for this path

def advanced_request(text, task):
    if not user_has_consented_to_cloud():
        return on_device_model.best_effort(text, task)   # degrade gracefully, stay local
    anonymized = strip_identifiers(text)
    return cloud_model.generate(anonymized, task)   # explicit opt-in only
```

PREP Interview Angle

Q: How do you deliver competitive GenAI quality on-device when the model has to be small enough to fit on a phone? SENIOR

Distillation and task-specific fine-tuning do most of the work here, compressing capability into one narrow, well-defined task instead of trying to replicate a general-purpose large model. Pair that with aggressive INT4/INT8 quantization tuned to the actual silicon, the Neural Engine, and build a consent-gated escalation path to the cloud that only kicks in for the rare request the on-device model genuinely can't handle.

> **Why this lands well at Apple:** Privacy-by-design with a graceful cloud-escalation path is exactly the architecture their ML interviews expect. It shows you'd default to on-device first, rather than treating the cloud as the obvious answer.

PYTHON Python Interview Question

Q: How do you strip identifying information from text in Python before any cloud escalation? SENIOR

A mix of regex and NER-based PII detection, something like Microsoft Presidio, to redact names, emails, and numbers, replacing each with a typed placeholder like [NAME] so the cloud model still has enough structure to be useful. Map the placeholders back onto the response afterward, and unit-test the redaction function against an adversarial set of edge cases before trusting it in production.

BONUS • PORTFOLIO

Top Projects to Showcase on Your Resume/GitHub

Beyond the 5 practice builds: a handful of projects that reliably get an interviewer's attention.

PORTFOLIO Ranked by Interview Impact

Project	Why It Impresses
End-to-end RAG system with an eval harness (RAGAS/DeepEval) in CI	Treats GenAI quality as something measurable, not a gut feeling
Multi-agent LangGraph app with human-in-the-loop approval	Agentic AI and production safety awareness, together
Fine-tuned (LoRA) small model beating a prompted large model on cost/latency	A real before/after number is the strongest story you can tell
Self-hosted LLM serving with vLLM + load test results	Proves infra depth beyond just calling an API
Guardrails/safety layer with red-teaming test suite	Uncommon skill, and highly valued in regulated industries
Cost dashboard for a GenAI pipeline ($/request breakdown)	Shows an ownership mindset, not just modeling ability

TIPS How to Present Any Project in an Interview

- Walk through it as **Problem, Constraint, Decision, Trade-off, Result**, in that order, and land on a number at the end.
- Keep one **failure story** ready per project: what broke, how you found it, what you changed. Interviewers trust that more than a clean success.
- Know your project's **cost per request** and **p95 latency** without checking notes. Those two numbers say more about production maturity than anything else you can mention.

PREP Interview Q&A

Q: Walk me through a project on your resume end to end. SENIOR

I keep it to about 90 seconds: the business problem, why I picked RAG, agents, or fine-tuning over the alternatives given the constraints I had, one specific hard problem I ran into in production, how I tracked it down and fixed it, and what actually improved as a result. After that I just let the interviewer steer toward whatever part they want to dig into.

PYTHON Python Interview Q&A

Q: What Python testing practices do you showcase in a portfolio project? SENIOR

Unit tests around the pure logic, things like prompt builders and output parsers, using pytest with the LLM calls mocked so the suite runs fast and doesn't flake. I'll add a small number of real integration tests behind a flag, plus coverage reporting in CI. A repo with a green test badge tells an interviewer more than a polished README ever will.

Q: How do you structure a Python repo for an LLM project so it reads as production-grade, not a notebook? SENIOR

Split it into clear modules, ingestion, retrieval, generation, api, keep configuration in environment variables rather than hardcoded keys, and pin dependencies in a requirements.txt or pyproject.toml. Add a README with an architecture diagram and setup steps, and wrap the entry point in a FastAPI app instead of leaving it as a loose script. That structure alone signals you've shipped something real before.

FINAL PAGE • RAPID FIRE

Basic LLM Interview Questions

The foundational stuff you need to answer without hesitating, since this is usually the screening round.

Q: What is an LLM?
A large neural network, usually a Transformer, trained on massive amounts of text to predict and generate human-like language.

Q: What is a token?
A sub-word unit of text that the model actually processes. Roughly 4 characters, or about 0.75 words, in English.

Q: What is context window?
The total number of tokens, input plus output combined, that a model can handle in a single call.

Q: What is temperature?
A setting that controls how random the output is. Close to 0 gives you deterministic, factual answers; higher values push toward more creative or varied ones.

Q: What is RAG?
Retrieval-Augmented Generation. You retrieve relevant documents and feed them into the prompt so the model's answer is grounded in real data rather than just what it memorized during training.

Q: What is fine-tuning?
Taking a pre-trained model and training it further on a smaller, task-specific labeled dataset to specialize how it behaves.

Q: What is LoRA?
Low-Rank Adaptation, a fine-tuning method that only trains small low-rank matrices instead of the full model, which cuts compute and memory cost dramatically.

Q: What is a vector database?
A database built to store embeddings and run fast approximate nearest-neighbor searches over them.

Q: What is prompt engineering?
Designing the structure, examples, and instructions in a prompt so you can reliably steer what a model outputs, without ever touching its weights.

Q: What is hallucination?
When a model produces something that sounds plausible but is factually wrong or has no real support behind it.

Q: What is an AI agent?
A system built on an LLM that can reason about a goal, decide which tools or actions to use, look at the results, and adjust, instead of just returning one answer and stopping.

Q: What is function/tool calling?
A model's ability to output a structured request to call an external function or API, then use whatever comes back as new information.

Q: Encoder vs decoder model?
Encoders build a bidirectional understanding of the input, which is good for classification. Decoders generate text left to right, one token at a time, which is what generation needs. GPT-class models are decoder-only.

Q: What is embedding similarity search based on?
Usually cosine similarity or dot product between the dense vector representations of two pieces of text.

Q: What is RLHF?
Reinforcement Learning from Human Feedback. You train a reward model on human-ranked responses, then use that to push the LLM's outputs toward what people actually prefer.

Q: What causes LLM inference latency?
Mostly autoregressive generation, since tokens come out one at a time, plus model size, context length, and how well the hardware batches requests.

One closing tip: after you give a crisp definition, add a sentence on *why it actually matters in production*. That's usually the difference an interviewer notices between someone who memorized definitions and someone who's actually shipped this stuff. Good luck, Deepika. You've got real production experience behind every one of these answers.